

Алгоритмы и структуры данных

Лекция 2. Линейные структуры данных

Ключарёв Пётр Георгиевич
pgkl@yandex.ru
twitter.com/klyucharev

Кафедра ИУ8 «Информационная безопасность»
МГТУ им. Н.Э. Баумана

2013 г.

Массивы

Массив

Массив занимает непрерывную область памяти.

Многомерный массив — это вид одномерного массива.

Сложность основных операций для массива длины n

- Изменение элемента (по индексу): $O(1)$.
- Вставка элемента (в произвольное место): $O(n)$.
- Удаление элемента: $O(n)$.
- Поиск элемента: $O(n)$.
- Доступ по индексу: $O(1)$.
- Дополнительный расход памяти: 0.

Массивы: достоинства и недостатки

Достоинства массивов

- Постоянное время доступа по индексу.
- Эффективное использование памяти.
- Локальность.

Недостатки массивов

- Невозможность изменения размера в процессе работы программы.
- Большое время поиска.

Динамический массив

Массив переменной длины.

$a.size$ — число элементов в динамическом массиве a .

$a.capacity$ — объем выделенной памяти (в элементах).

Алгоритм 1 Добавление элемента x в конец дин. массива a

```
1: procedure INSERTEND( $a, x$ )
2:   if  $a.size = a.capacity$  then
3:     Выделить память  $k * a.capacity$ . //  $O(1)$ 
4:      $a.capacity := k * a.capacity$ 
5:     Скопировать старый массив в новый. //  $O(n)$ 
6:     Освободить память. //  $O(1)$ 
7:    $a[a.size] := x$ 
8:    $a.size := a.size + 1$ 
```

Сложность добавления элемента в дин. массив

Сложность в худшем случае

$$T(n) = O(n),$$

где n — число элементов в массиве.

Сложность в среднем

Начнем с пустого массива и последовательно добавим n элементов. При этом половина элементов перемещается один раз, четверть — два раза, и т.д. Общее число перемещений:

$$M = \sum_{i=1}^{\log_2 n} \frac{i \cdot n}{2^i} < n \sum_{i=1}^{\infty} \frac{i}{2^i} = 2n.$$

Средняя сложность:

$$T_m(n) = \frac{k \cdot M}{n} = O(1).$$

Стек

Стек

Стек — это структура данных, поддерживающая две операции:

- *push* — добавление элемента;
- *pop* — получение элемента и удаление его из стека.

При этом *pop* выдает элемент, добавленный последнем (поэтому стек называют LIFO).

Стек на основе массива

Массив $S[1..n]$. Наряду с ним будем хранить число $S.top$ — индекс последнего добавленного в стек элемента.

$S[1]$ — нижний элемент стека(дно).

$S[S.top]$ — верхний элемент стека (вершина).

Если $S.top = 0$, то стек пуст. Если $S.top = n$, то попытка добавления элемента приводит к переполнению.

Стек

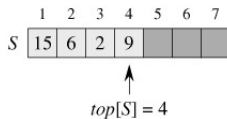
Алгоритм 2 Добавление элемента

```
1: procedure PUSH( $S, x$ )  
2:    $S.top := S.top + 1$   
3:    $S[S.top] := x$ 
```

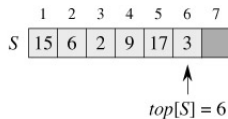
Алгоритм 3 Получить элемент

```
1: procedure POP( $S$ )  
2:   if  $S.top = 0$  then  
3:     error “underflow”  
4:   else  
5:      $S.top := S.top - 1$   
6:     return  $S[S.top + 1]$ 
```

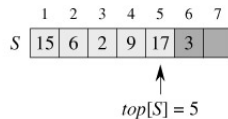
Стек



(a)



(b)



(c)

Очередь

Очередь

Очередь — это структура данных, поддерживающая две операции:

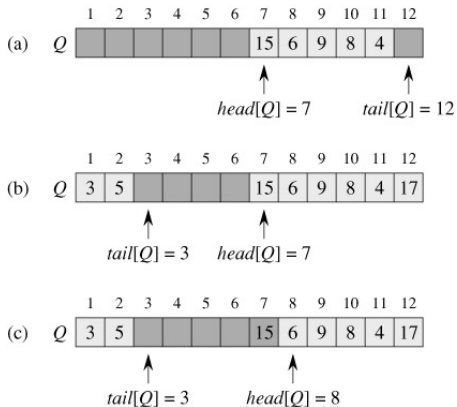
- *enqueue* — добавление элемента;
- *dequeue* — получение элемента и удаление его из очереди.

При этом, *dequeue* выдает элемент, добавленный первым (поэтому очередь называют FIFO).

Очередь на основе массива

У очереди Q есть голова (элемент с индексом $Q.head$) и хвост (элемент с индексом $Q.tail$). Очередь состоит из элементов массива с индексами $Q.head, Q.head + 1, \dots, Q.tail - 1$ (Предполагается, что массив свернут в кольцо).

Очередь



Основные операции с очередью

Алгоритм 4 Добавление элемента x в очередь Q

```
1: procedure ENQUEUE( $Q, x$ )  
2:    $Q[Q.tail] := x$   
3:   if  $Q.tail = Q.length$  then //  $Q.length$  — длина массива.  
4:      $Q.tail := 1$   
5:   else  
6:      $Q.tail := Q.tail + 1$ 
```

Основные операции с очередью

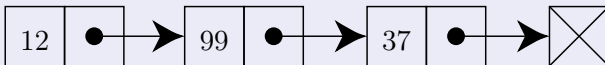
Алгоритм 5 Получить элемент из очереди Q

```
1: procedure DEQUEUE( $Q$ )  
2:    $x := Q[Q.head]$   
3:   if  $Q.head = Q.length$  then  
4:      $Q.head := 1$   
5:   else  
6:      $Q.head := Q.head + 1$   
7:   return  $x$ 
```

СВЯЗНЫЙ СПИСОК

Связный список

Структура данных состоящая из узлов, каждый из которых содержит как данные, так и ссылку на следующий элемент.



Обозначения

Элемент списка содержит два поля: ссылку на следующий элемент и данные.

Пусть x — элемент списка.

$x.next$ — следующий элемент.

$x.key$ — полезные данные (ключ).

Пустую ссылку будем обозначать nil .

$L.head$ — голова списка L (т.е. первый элемент). Если $L.head = nil$, то список пуст.

СВЯЗНЫЙ СПИСОК

Алгоритм 6 Поиск в списке

```
1: procedure LIST-SEARCH( $L, key$ )  
2:    $t := L.head$   
3:   while  $t \neq nil$  and  $t.key \neq key$  do  
4:      $t := t.next$   
5:   return  $t$ 
```

Алгоритм 7 Добавление элемента в список перед первым элементом

```
1: procedure LIST-INSERTBEGINNING( $L, x$ )  
2:    $x.next := L.head$   
3:    $L.head := x$ 
```

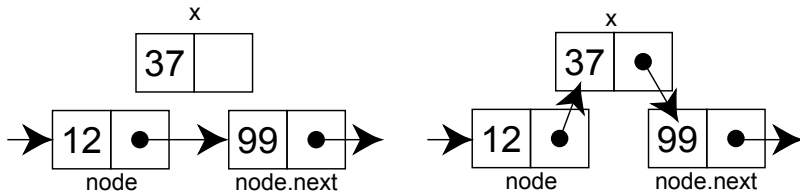
СВЯЗНЫЙ СПИСОК

Алгоритм 8 Добавление элемента в список после элемента *node*

1: **procedure** LIST-INSERTAFTER(*node*, *x*)

2: *x.next* := *node.next*

3: *node.next* := *x*



СВЯЗНЫЙ СПИСОК

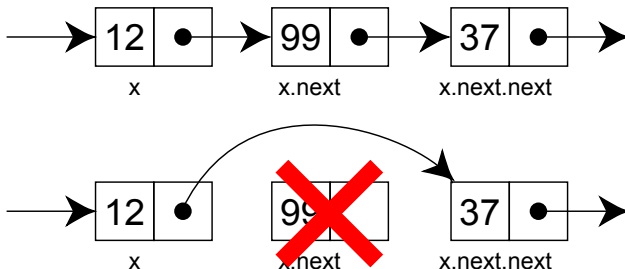
Алгоритм 9 Удаление элемента, следующего за данным

1: **procedure** LIST-REMOVEAFTER(x)

2: $t := x.next$

3: $x.next := x.next.next$

4: Удалить t из памяти.



Алгоритм 10 Удаление первого элемента из списка

- 1: **procedure** LIST-REMOVEBEGINNING(L)
 - 2: $t := L.head$
 - 3: $L.head := t.next$
 - 4: Удалить t из памяти.
-

Сложность операций над связным списком

Поиск: $O(n)$.

Добавление элемента в начало списка: $O(1)$.

Добавление элемента (после данного): $O(1)$.

Добавление в остальных случаях: $O(n)$

Удаление элемента из начала списка: $O(1)$.

Удаление элемента (если есть ссылка на предыдущий): $O(1)$.

Удаление в остальных случаях: $O(n)$.

Доступ по индексу: $O(n)$.

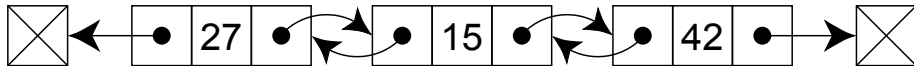
Здесь n — длина списка.

Двусвязный список

Двусвязный список

Каждый элемент такого списка содержит:

- Данные *key*.
- Ссылку на предыдущий элемент *prev*.
- Ссылку на следующий элемент *next*.



Двусвязный список: основные операции

Алгоритм 11 Добавление элемента в двусвязный список перед первым элементом

```
1: procedure DOUBLYLIST-INSERTBEGINNING( $L, x$ )  
2:    $x.next := L.head$   
3:   if  $L.head \neq nil$  then  
4:      $L.head.prev := x$   
5:    $L.head := x$   
6:    $x.prev := nil$ 
```

Двусвязный список: основные операции

Алгоритм 12 Добавление элемента x в двусвязный список после элемента $node$

```
1: procedure DOUBLYLIST-INSERTAFTER( $node, x$ )  
2:    $x.prev := node$   
3:    $x.next := node.next$   
4:    $node.next := x$   
5:   if  $x.next \neq nil$  then  
6:      $x.next.prev := x$ 
```

Двусвязный список: основные операции

Алгоритм 13 Удаление элемента x из двусвязного списка L

```
1: procedure DOUBLYLIST-REMOVE( $L, x$ )  
2:   if  $x.prev \neq nil$  then  
3:      $x.prev.next := x.next$   
4:   else  
5:      $L.head := x.next$   
6:   if  $x.next \neq nil$  then  
7:      $x.next.prev := x.prev$   
8:   Удалить  $x$  из памяти.
```

Списки: достоинства и недостатки

Достоинства

- Переменный размер.
- Быстрая вставка и удаление элементов.

Недостатки

- Повышенный расход памяти.
- Большое время поиска.

Упражнения

Добавление проверок

Добавить проверки на переполнение в алгоритмы работы с очередью.

Стек и очередь на основе списков

Построить стек и очередь на основе связанных списков.

Список с фиктивным элементом

Пусть в списке имеется фиктивный элемент, играющий роль ссылки *nil*. Будем обозначать его *L.nil*. *L.nil.next* указывает на первый элемент списка. *L.nil.prev* указывает на последний элемент списка. Построить алгоритмы для:

- Поиска элемента в таком списке.
- Добавления элемента.
- Удаления элемента.

И еще упражнения...

Обращение списка

Написать процедуру, которая переставляет элементы списка в обратном порядке за время $\Theta(n)$. Объем дополнительной памяти должен быть $O(1)$.

Операции с отсортированным массивом

Найти сложность основных операций над отсортированным массивом (поиск, добавление элемента, удаление элемента, нахождение максимума и минимума). Сравнить со сложностью этих операций над неотсортированным массивом.